



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №12

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 Работа с JSON	3
1.2 Провайдеры контента	7
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	10
2.1 Реализация передачи данных в другое приложение через провайдер контента.....	10
2.2 Работа с JSON файлом.....	15
ЗАКЛЮЧЕНИЕ	18

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 Работа с JSON

JSON (JavaScript Object Notation) — это легкий формат обмена данными, основанный на подмножестве языка JavaScript. JSON используется для передачи структурированных данных между клиентом и сервером. Он часто применяется в вебразработке, мобильных приложениях и других системах, где требуется передача данных.

JSON состоит из пар ключ-значение, где ключи обычно строковые, а значения могут быть строками, числами, логическими значениями, массивами, объектами или null. Его легко читать и понимать как человеку, так и компьютеру, что делает его идеальным для обмена данными.

В Android разработке JSON часто используется для передачи данных между клиентом и сервером или для сохранения информации локально.

Для работы с форматом json нет встроенных средств, но есть куча библиотек и пакетов, которые можно использовать для данной цели. Одним из наиболее популярных из них является пакет `com.google.code.gson`.

Для облегчения работы с JSON объектами с использованием данного пакета необходимо дополнительно добавить библиотеку Gson в `build.gradle.kts` (рисунок 1.1.1).

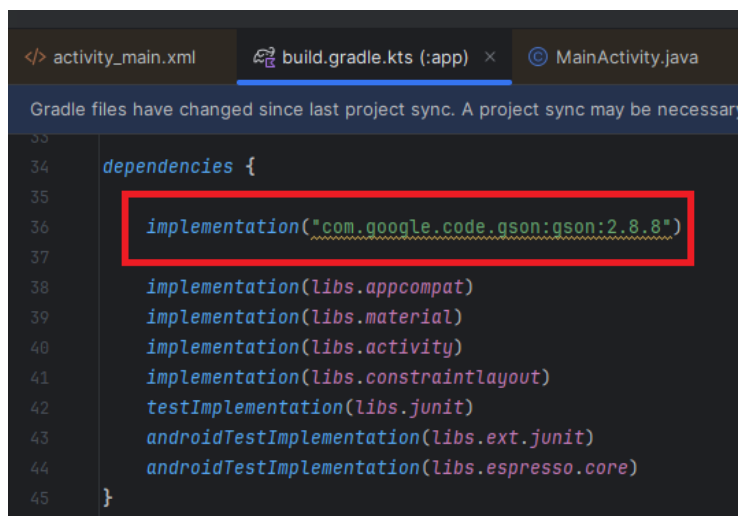
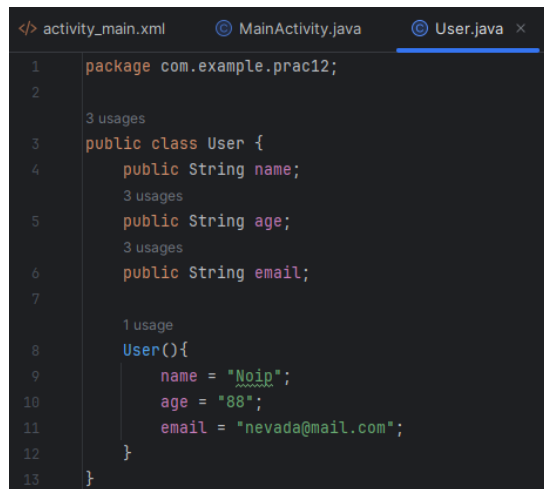


Рисунок 1.1.1 – Добавление зависимостей в сборку приложения

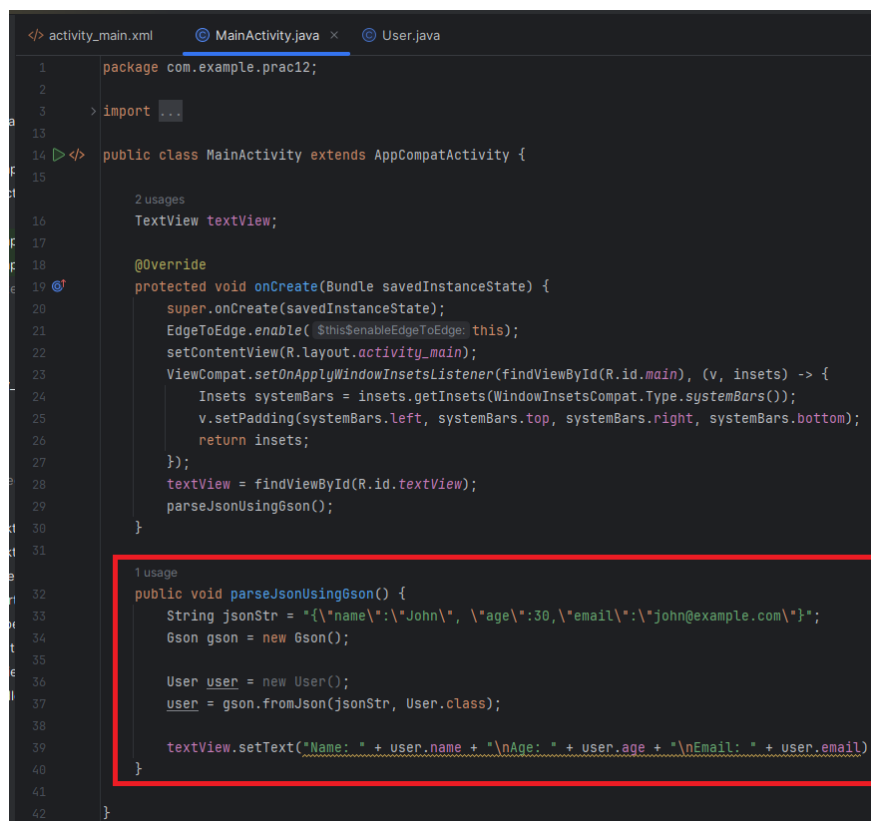
Теперь рассмотрим несколько примеров работы с JSON и классом User, состоящим из полей: имя, возраст, почта (рисунок 1.1.2).



```
<> activity_main.xml  MainActivity.java  User.java x
1 package com.example.prac12;
2
3 usages
4 public class User {
5     public String name;
6     public String age;
7     public String email;
8
9     1 usage
10    User(){
11        name = "Noip";
12        age = "88";
13        email = "nevada@mail.com";
14    }
15 }
```

Рисунок 1.1.2 – Описание класса User

Далее необходимо создать JSON строки и десериализовать её данные (рисунок 1.1.3).



```
<> activity_main.xml  MainActivity.java x  User.java
1 package com.example.prac12;
2
3 > import ...
4
5 public class MainActivity extends AppCompatActivity {
6
7     2 usages
8     TextView textView;
9
10
11     @Override
12     protected void onCreate(Bundle savedInstanceState) {
13         super.onCreate(savedInstanceState);
14         EdgeToEdge.enable( $this$enableEdgeToEdge: this);
15         setContentView(R.layout.activity_main);
16         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
17             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
18             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
19             return insets;
20         });
21         textView = findViewById(R.id.textView);
22         parseJsonUsingGson();
23     }
24
25     1 usage
26     public void parseJsonUsingGson() {
27         String jsonStr = "{\"name\":\"John\", \"age\":30, \"email\":\"john@example.com\"}";
28         Gson gson = new Gson();
29
30         User user = new User();
31         user = gson.fromJson(jsonStr, User.class);
32
33         textView.setText("Name: " + user.name + "\nAge: " + user.age + "\nEmail: " + user.email);
34     }
35 }
```

Рисунок 1.1.3 – Создание JSON строки и её десериализация

При запуске приложения с activity, изображённой на рисунке 1.1.3, на экране пользователя отобразятся полученные при десериализации данные (рисунок 1.1.4).

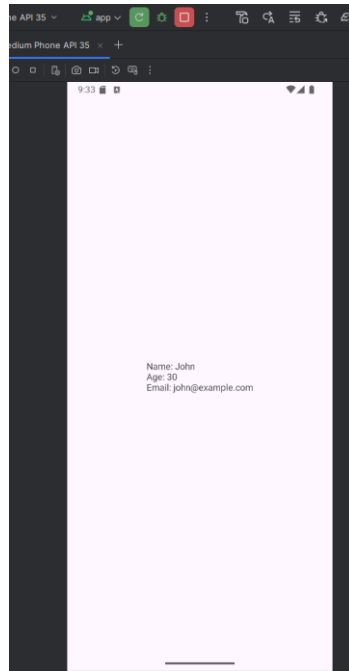


Рисунок 1.1.4 – Отображение десериализованной JSON строки в итоговом приложении

Для работы с json создается объект Gson. Для десериализации выполняется метод `fromJson()`, в который передается объект класса User с сериализованными данными и тип, к которому надо десериализовать данные.

Для создания json строки необходимо воспользоваться методом `toJson()`, в который подаются сериализуемые данные и который в данном случае возвратит json строку (рисунок 1.1.5).

```
no usages
public String createJsonUsingGson() {
    User user = new User();
    user.name = "Alice";
    user.age = "25";
    user.email = "alice@example.com";

    Gson gson = new Gson();
    return gson.toJson(user);
}
```

Рисунок 1.1.5 – Метод создания одной json строки

Для чтения же JSON массива потребуется метод `fromJson()` (рисунок 1.1.6).

```

1 usage
public void parseJsonArrayUsingGson() {
    String jsonArrayStr = "[{\"name\":\"John\", \"age\":30,\"email\":\"john@example.com\"}, " +
        "{\"name\":\"Alice\", \"age\":25,\"email\":\"alice@example.com\"}]";
    Gson gson = new Gson();
    Type userListType = new TypeToken<List<User>>().getType();

    List<User> users = gson.fromJson(jsonArrayStr, userListType);

    for (User user : users) {
        textView.setText(textView.getText() + "\nName: " + user.name + "\nAge: " + user.age + "\nEmail: " + user.email + "\n---");
    }
}

```

Рисунок 1.1.6 – Метод чтения JSON массива

При запуске приложения с вызовом метода, представленного на рисунке 1.1.6, пользователю отобразятся десериализованные данные (рисунок 1.1.7).

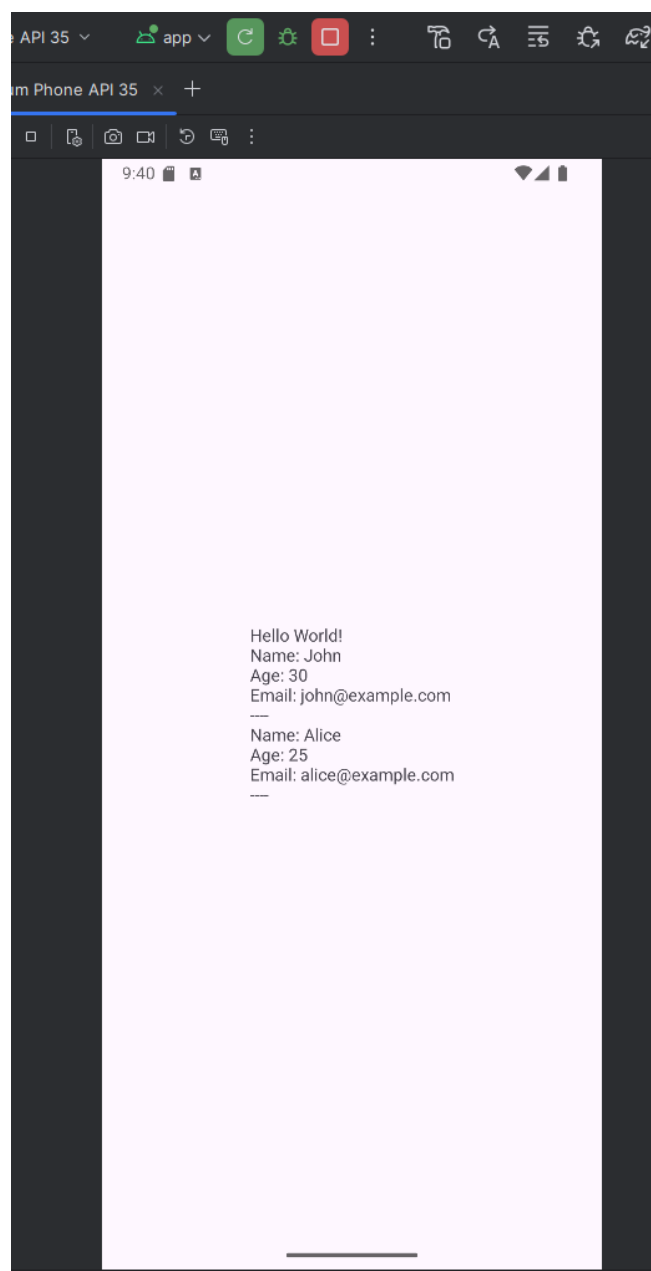


Рисунок 1.1.7 – Итоговый вид приложения

1.2 Провайдеры контента

Провайдеры контента (Content Providers) в Android представляют собой компоненты, которые позволяют приложениям делиться данными между собой. Провайдеры контента инкапсулируют данные и предоставляют механизмы для определения политик доступа к этим данным. Это особенно полезно в среде, где множество различных приложений пытаются работать с одним и тем же набором данных или когда данные должны быть предоставлены из одного приложения другим.

Провайдеры контента в Android предоставляют следующие преимущества:

- безопасный доступ к данным: провайдеры контента могут ограничивать доступ к данным приложения, предоставляя только необходимые данные и скрывая внутреннюю структуру базы данных;
- стандартизация доступа к данным: они предлагают унифицированный интерфейс для работы с данными, что упрощает обмен данными между приложениями;
- управление данными: провайдеры контента упрощают управление данными, так как обработка запросов, вставка, удаление и обновление данных производятся через стандартизированные API;
- интеграция с Android системой: провайдеры могут легко интегрироваться с другими Android компонентами, такими как загрузчики (Loaders) и система поиска.

Рассмотрим следующий пример с провайдером контента.

Имеется приложение по работе с коллекцией книг, хранящейся в базе данных SQLite и вы хотите предоставить другим приложениям доступ к информации о книгах пользователя.

Для начала необходимо определить URI (Uniform Resource Identifier) — уникальный идентификатор, используемый для идентификации данных в провайдере контента (рисунок 1.2.1). В Android URI для провайдера

контента обычно включает в себя authority, который является символьным идентификатором провайдера.

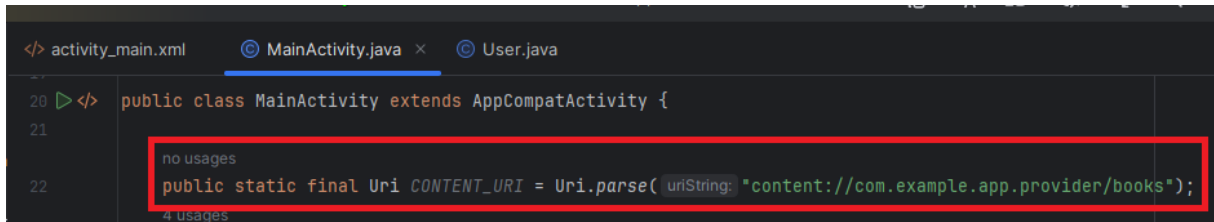


Рисунок 1.2.1 – Объявление и инициализация поля URI

Далее необходимо реализовать класс провайдера и добавить в него базу данных.

Для создания собственного контент-провайдера нужно унаследоваться от абстрактного класса ContentProvider (рисунок 1.2.2).

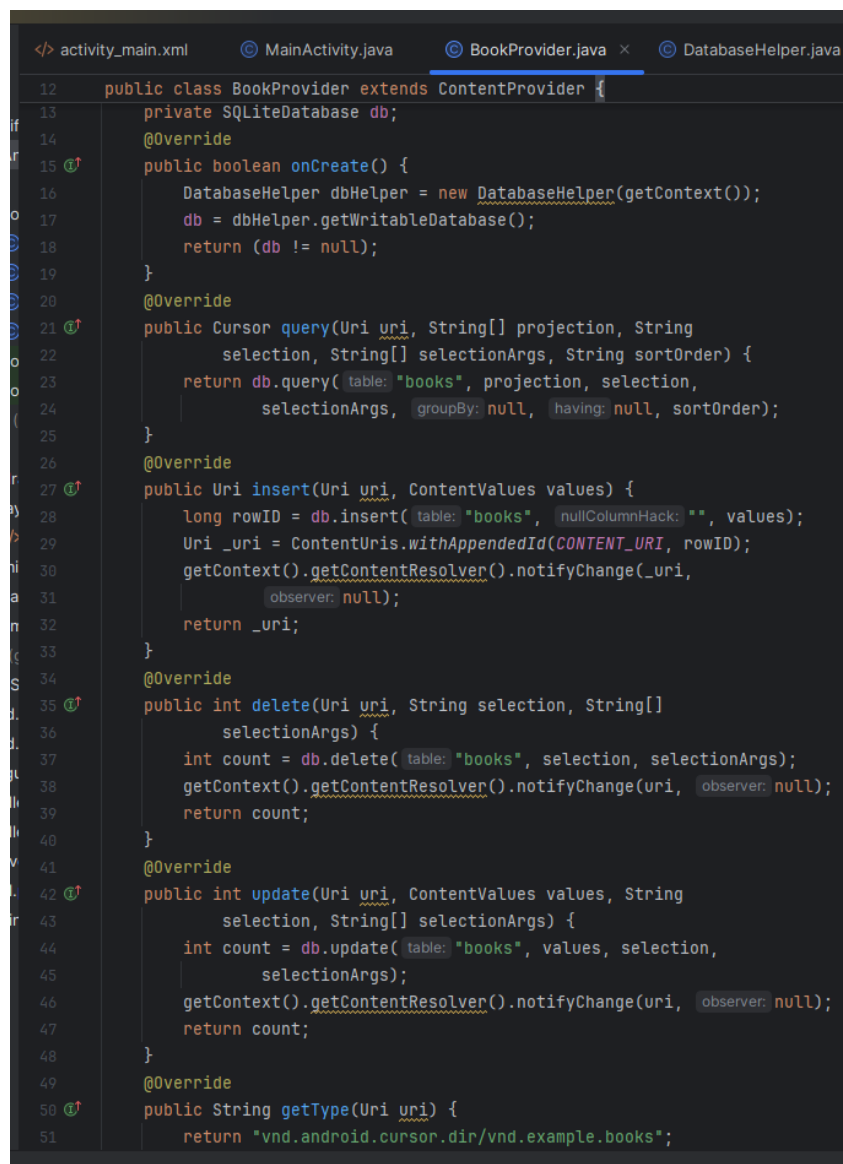
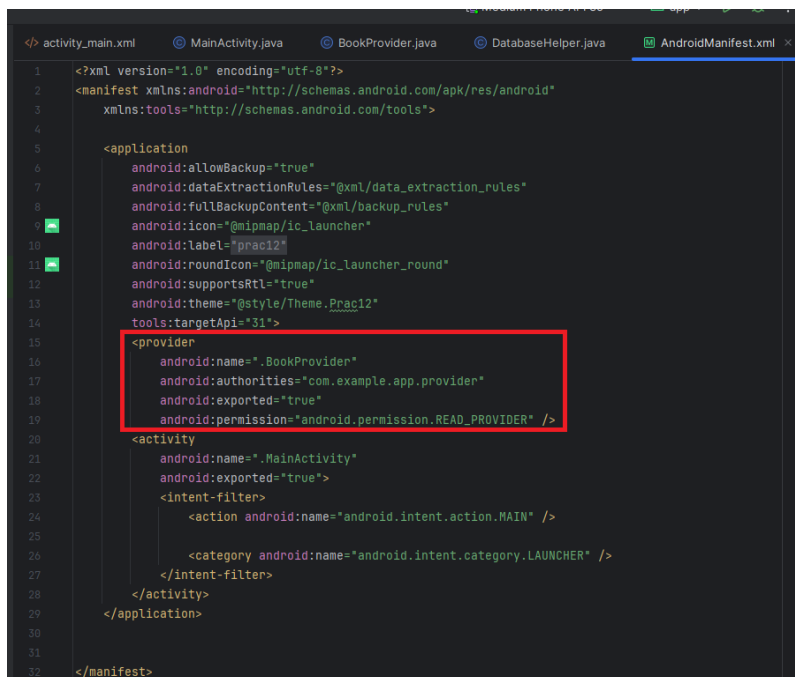


Рисунок 1.2.2 – Описание класса BookProvider

Последним шагом станет добавление провайдера в манифесте (рисунок 1.2.3) и заполнение базы данных информацией о книгах.

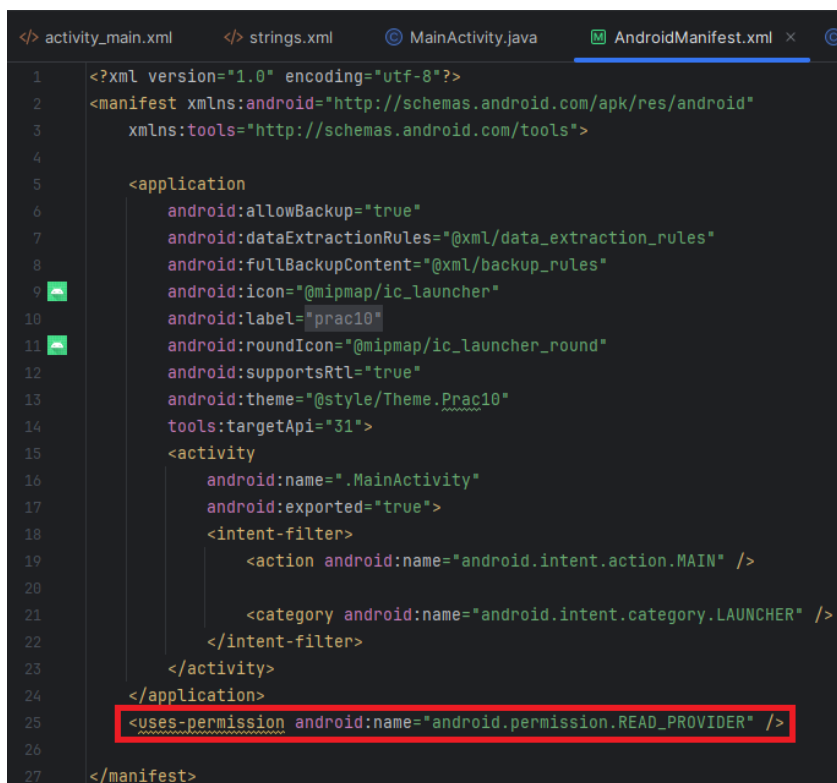


```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/prac12"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Prac12"
        tools:targetApi="31">
        <provider
            android:name=".BookProvider"
            android:authorities="com.example.app.provider"
            android:exported="true"
            android:permission="android.permission.READ_PROVIDER" />
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Рисунок 1.2.3 – Описание манифеста приложения, записывающего данные в провайдер

Также в манифесте другого приложения даем разрешение на чтение данных из провайдера (рисунок 1.2.4).



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/prac10"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Prac10"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
    <uses-permission android:name="android.permission.READ_PROVIDER" />
</manifest>
```

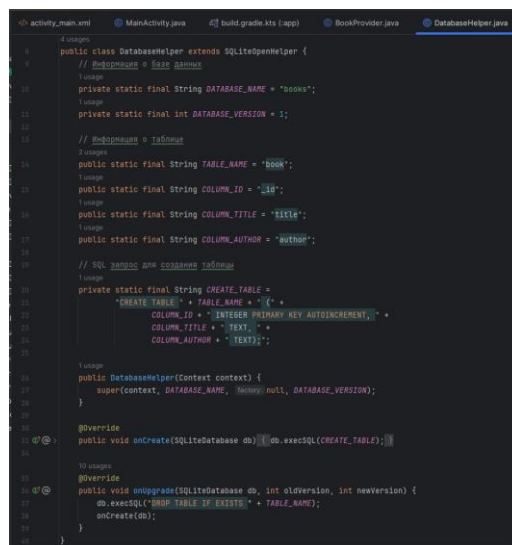
Рисунок 1.2.4 - Описание манифеста приложения, считывающего данные из провайдера

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Реализация передачи данных в другое приложение через провайдер контента

Для начала опишем приложение, записывающее данные в провайдер контента.

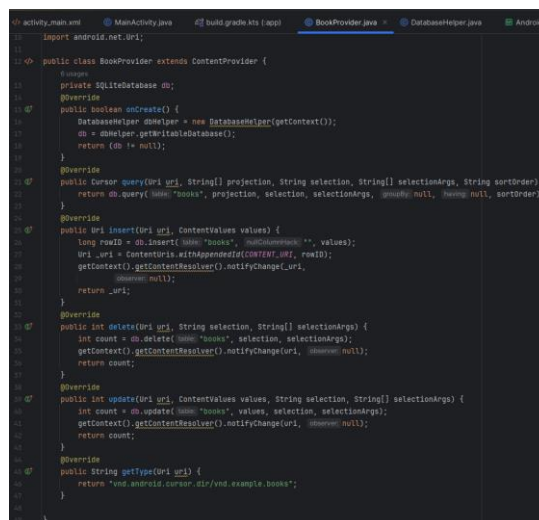
Для этого необходимо описать логику класса DatabaseHelper (рисунок 2.1.1).



```
1 public class DatabaseHelper extends SQLiteOpenHelper {
2     // @formatter:off
3     // @formatter:on
4     private static final String DATABASE_NAME = "books";
5     private static final int DATABASE_VERSION = 1;
6
7     // @formatter:off
8     // @formatter:on
9     public static final String TABLE_NAME = "BOOK";
10    private static final String COLUMN_ID = "_id";
11    private static final String COLUMN_TITLE = "TITLE";
12    private static final String COLUMN_AUTHOR = "AUTHOR";
13
14    // SQL statements for creating the database
15    private static final String CREATE_TABLE =
16        "CREATE TABLE " + TABLE_NAME + " (" +
17        "    COLUMN_ID " + "INTEGER PRIMARY KEY AUTOINCREMENT," +
18        "    COLUMN_TITLE " + "TEXT," +
19        "    COLUMN_AUTHOR " + "TEXT)";
20
21    public DatabaseHelper(Context context) {
22        super(context, DATABASE_NAME, null, DATABASE_VERSION);
23    }
24
25    @Override
26    public void onCreate(SQLiteDatabase db) {
27        db.execSQL(CREATE_TABLE);
28    }
29
30    @Override
31    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
32        db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
33        onCreate(db);
34    }
35 }
```

Рисунок 2.1.1 – Описание класса DatabaseHelper

Класс DatabaseHelper необходим для создания базы данных, с которой будет взаимодействовать ContentProvider (рисунок 2.1.2).



```
1 import android.net.Uri;
2
3 public class BooksProvider extends ContentProvider {
4     private SQLiteDatabase db;
5
6     @Override
7     public boolean onCreate() {
8         DatabaseHelper dbHelper = new DatabaseHelper(getContext());
9         db = dbHelper.getWritableDatabase();
10        return (db != null);
11    }
12
13    @Override
14    public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder) {
15        return db.query(uri, "books", projection, selection, selectionArgs, null, null, sortOrder);
16    }
17
18    @Override
19    public Uri insert(Uri uri, ContentValues values) {
20        long rowId = db.insert(uri, "books", null, values);
21        Uri _uri = ContentUris.withAppendedId(CONTENT_URI, rowId);
22        getContext().getContentResolver().notifyChange(_uri, null);
23        return _uri;
24    }
25
26    @Override
27    public int delete(Uri uri, String selection, String[] selectionArgs) {
28        int count = db.delete(uri, "books", selection, selectionArgs);
29        getContext().getContentResolver().notifyChange(uri, null);
30        return count;
31    }
32
33    @Override
34    public int update(Uri uri, ContentValues values, String selection, String[] selectionArgs) {
35        int count = db.update(uri, "books", values, selection, selectionArgs);
36        getContext().getContentResolver().notifyChange(uri, null);
37        return count;
38    }
39
40    @Override
41    public String getType(Uri uri) {
42        return "vnd.android.cursor.dir/vnd.example.books";
43    }
44 }
```

Рисунок 2.1.2 – Описание класса BooksProvider

Класс, описанный на рисунке 2.1.2 реализует запись данных в провайдер контента.

Для работы записи данных в провайдер контента необходимо прописать разрешение на запись данных в провайдер в манифесте приложения (рисунок 2.1.3).

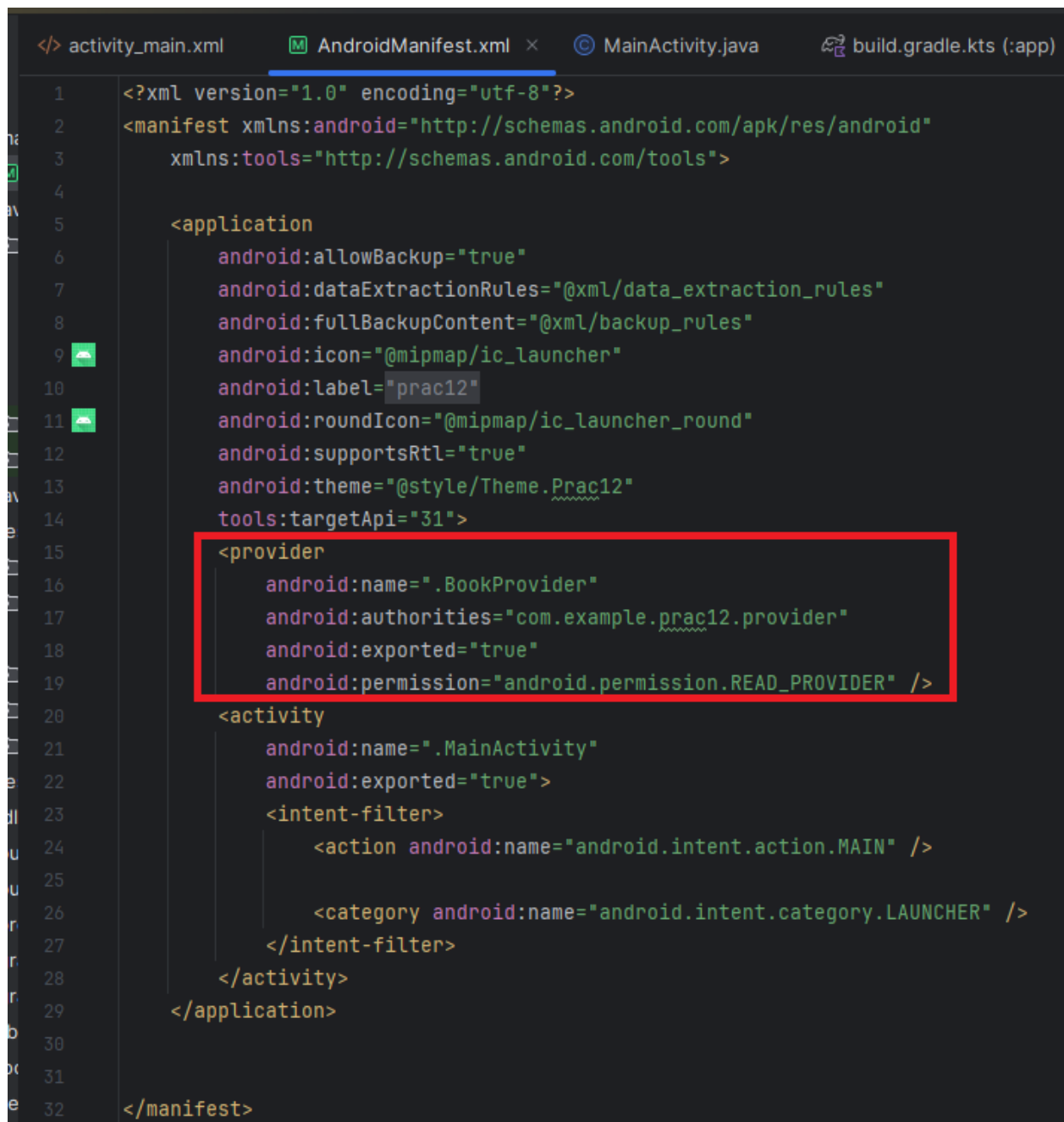


Рисунок 2.1.3 – Описание манифеста приложения

Далее необходимо описать разметку главной activity (рисунок 2.1.4).

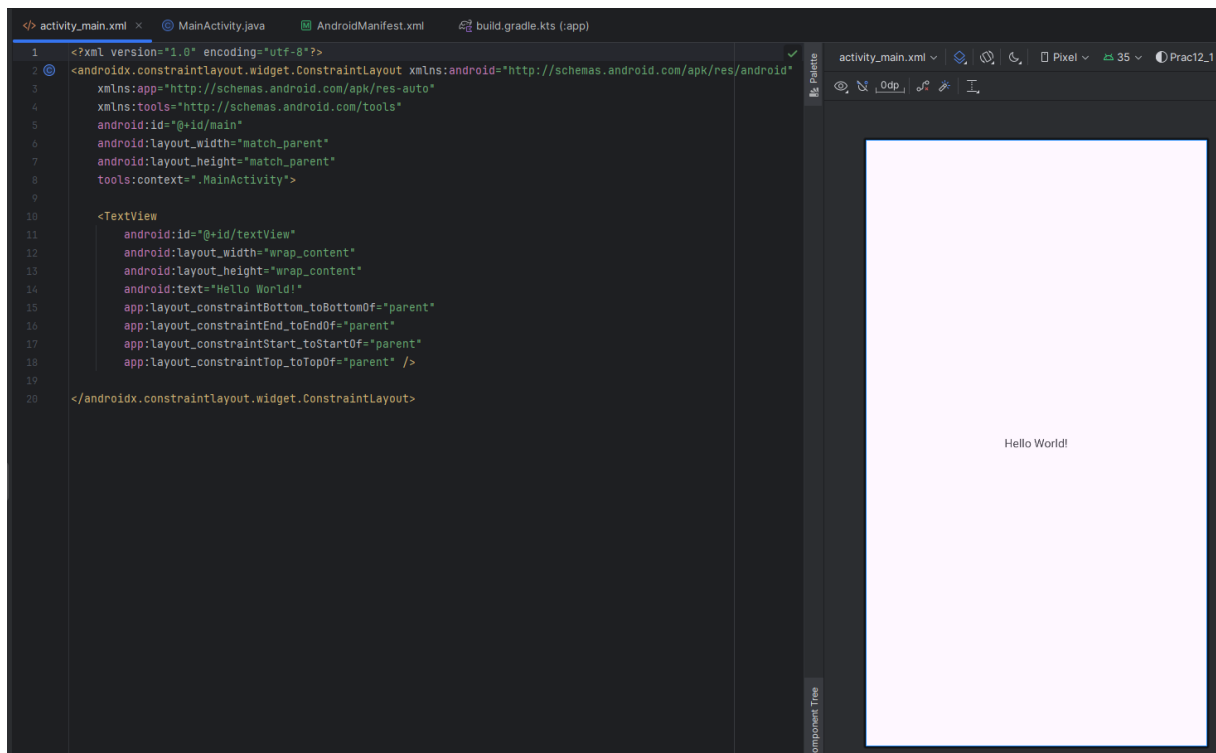


Рисунок 2.1.4 – Кодовое и графическое описание главной activity

Далее необходимо описать логику работу главной activity, через которую и будет производиться запись данных в провайдер контента при помощи метода `insert()` (рисунок 2.1.5).

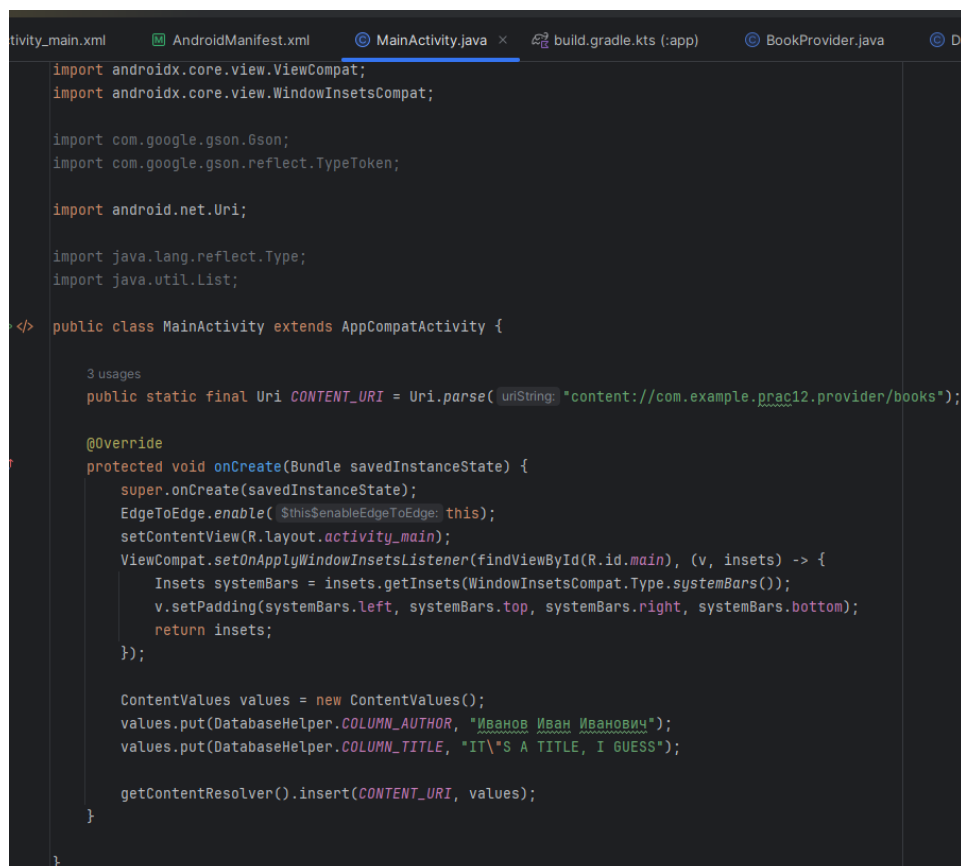
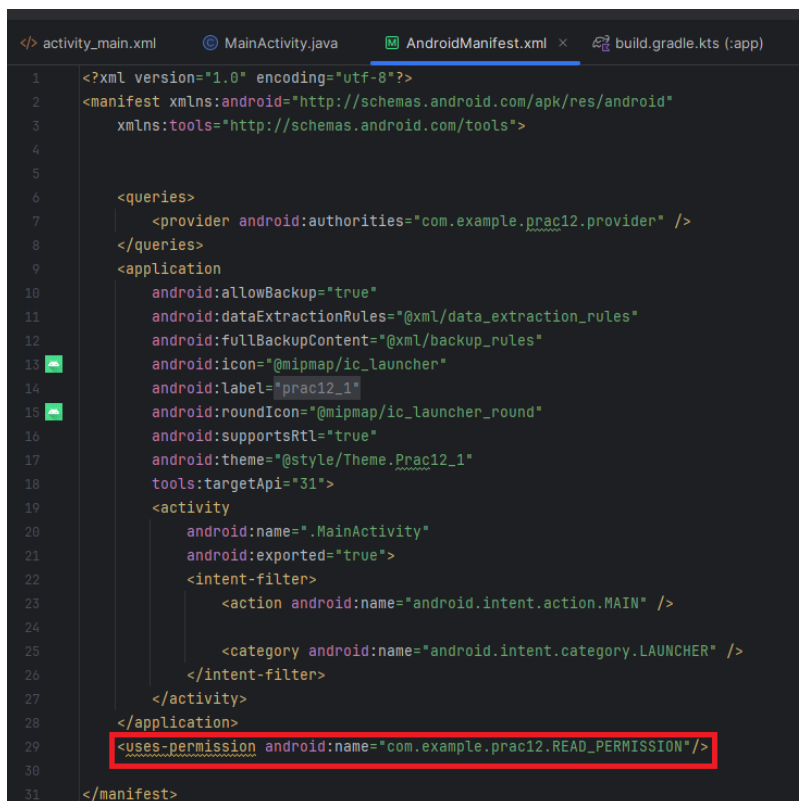


Рисунок 2.1.5 – Описание логики работы главной activity

При запуске приложения произойдёт запись базы данных “books” в провайдер контента.

Далее следует создание приложения, считывающего информацию из провайдера контента. Для этого необходимо прописать разрешение на считывание данных из провайдера контента (рисунок 2.1.6).



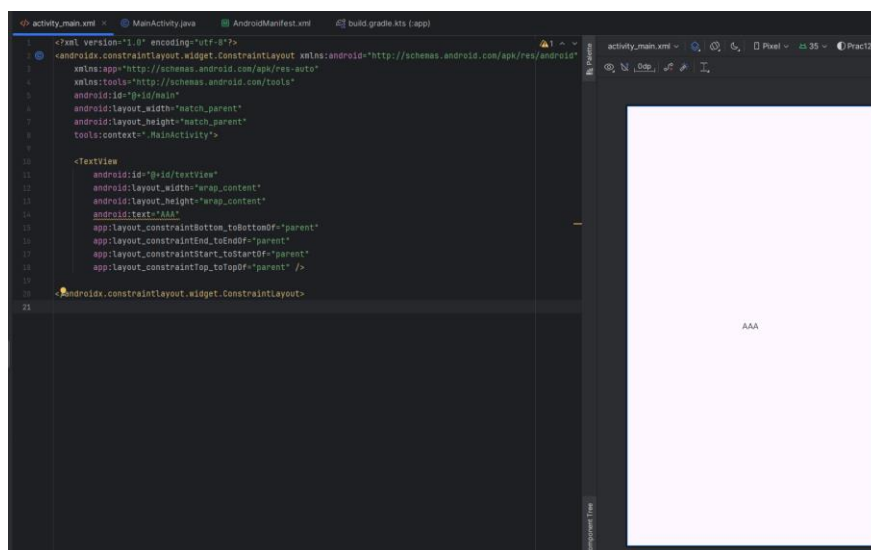
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <queries>
        <provider android:authorities="com.example.prac12.provider" />
    </queries>
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="prac12_1"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Prac12_1"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
    <uses-permission android:name="com.example.prac12.READ_PERMISSION"/>
</manifest>
```

Рисунок 2.1.6 – Описание манифеста приложения

Также необходимо описать файл разметки MainActivity с использованием TextView для отображения полученных данных.



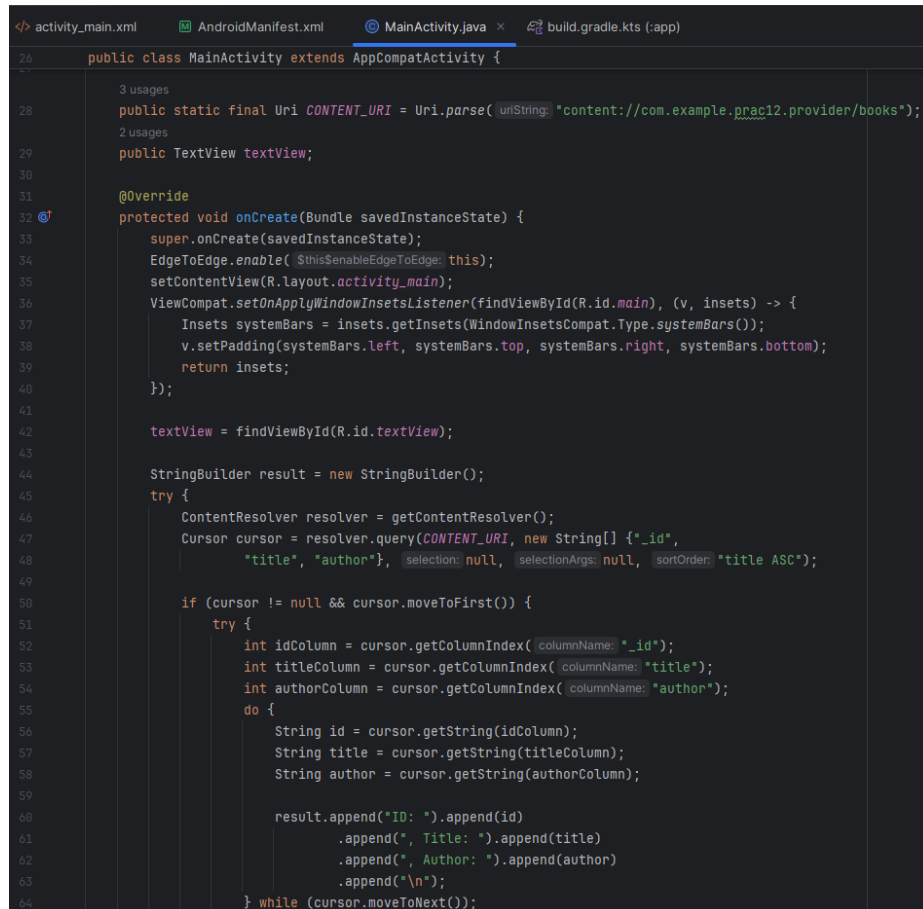
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textview"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="AAA"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Рисунок 2.1.7 – Описание файла разметки главной activity

Далее необходимо описать логику считывания данных из провайдера контента (рисунки 2.1.8 – 2.1.9).



```
26 public class MainActivity extends AppCompatActivity {
27
28     3 usages
29     public static final Uri CONTENT_URI = Uri.parse("content://com.example.prac12.provider/books");
30     2 usages
31     public TextView textView;
32
33     @Override
34     protected void onCreate(Bundle savedInstanceState) {
35         super.onCreate(savedInstanceState);
36         EdgeToEdge.enable(this);
37         setContentView(R.layout.activity_main);
38         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
39             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
40             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
41             return insets;
42         });
43
44         textView = findViewById(R.id.textview);
45
46         StringBuilder result = new StringBuilder();
47         try {
48             ContentResolver resolver = getContentResolver();
49             Cursor cursor = resolver.query(CONTENT_URI, new String[] { "_id",
50                 "title", "author" }, selection: null, selectionArgs: null, sortOrder: "title ASC");
51
52             if (cursor != null && cursor.moveToFirst()) {
53                 try {
54                     int idColumn = cursor.getColumnIndex(columnName: "_id");
55                     int titleColumn = cursor.getColumnIndex(columnName: "title");
56                     int authorColumn = cursor.getColumnIndex(columnName: "author");
57                     do {
58                         String id = cursor.getString(idColumn);
59                         String title = cursor.getString(titleColumn);
60                         String author = cursor.getString(authorColumn);
61
62                         result.append("ID: ").append(id)
63                             .append(", Title: ").append(title)
64                             .append(", Author: ").append(author)
65                             .append("\n");
66                     } while (cursor.moveToNext());
67                 }
68             }
69         } catch (Exception e) {
70             result.append("Нет данных или ошибка доступа");
71         }
72         textView.setText(result.toString());
73     }
74 }
```

Рисунок 2.1.8 – Часть 1 описания главной activity



```
        .append("\n");
    } while (cursor.moveToNext());
    } finally {
        cursor.close();
    }

    } else {
        result.append("Нет данных или ошибка доступа");
    }
} catch (Exception e) {
    result.append("Ошибка: ").append(e.getMessage());
}

textView.setText(result.toString());
}
```

Рисунок 2.1.9 – Часть 2 описания главной activity

На рисунках 2.1.8 – 2.1.9 описан процесс считывания данных из провайдера контента, а также отображение считанных данных на экран (рисунок 2.1.10).

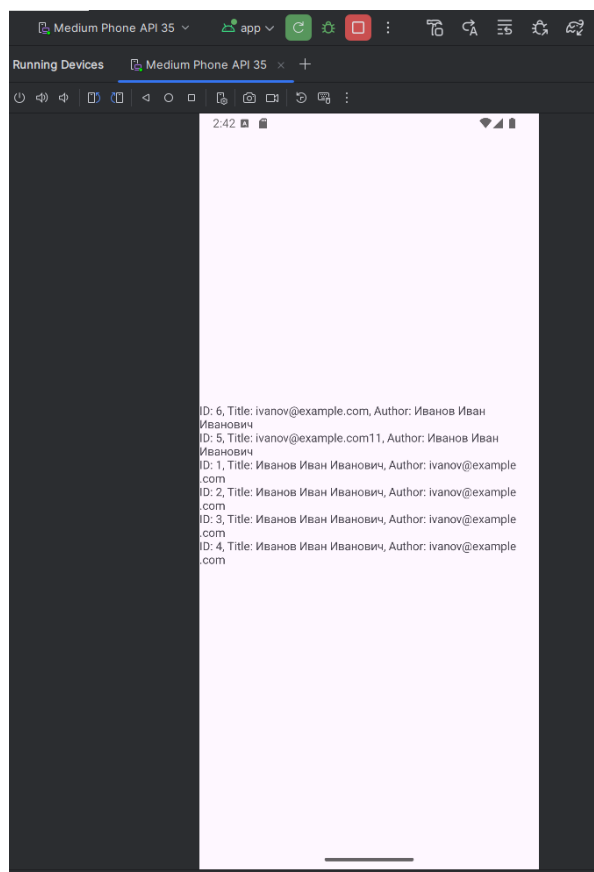


Рисунок 2.1.10 – Итоговый вид приложения

2.2 Работа с JSON файлом

Для начала необходимо описать файл разметки главной activity (рисунок 2.2.1).

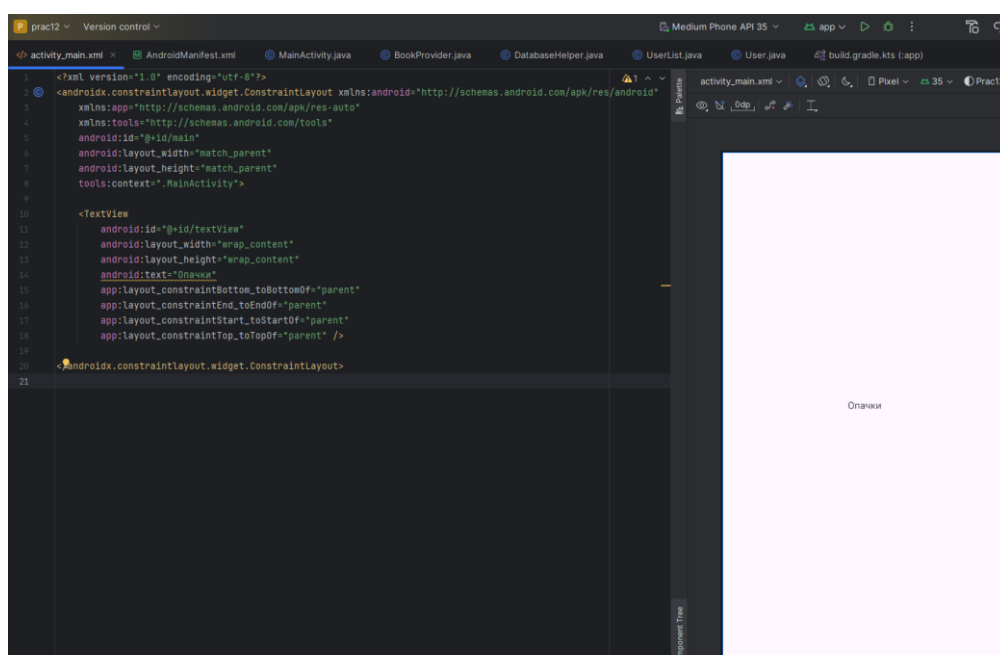
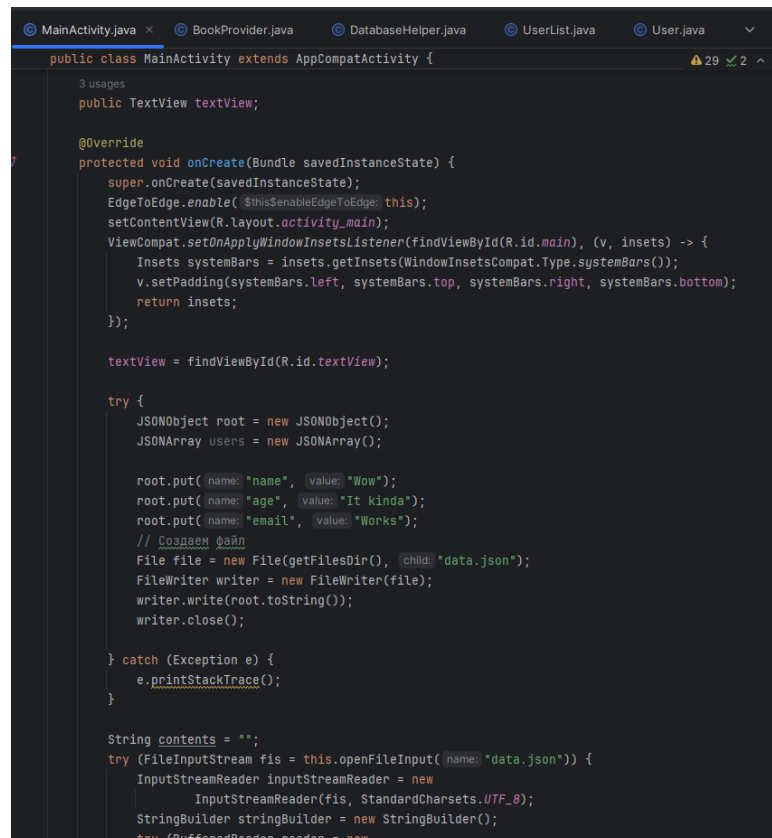


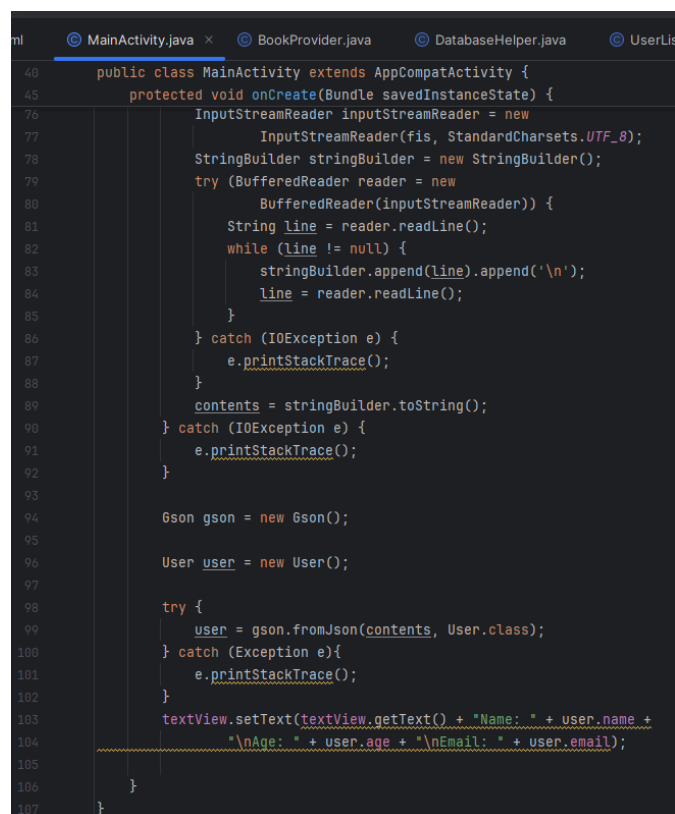
Рисунок 2.2.1 – Кодовый и графический вид итогового приложения

Далее необходимо описать запись, сохранение и чтение JSON файла (рисунки 2.2.2 – 2.2.3).



```
public class MainActivity extends AppCompatActivity {  
    3 usages  
    public TextView textView;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        EdgeToEdge.enable(this);  
        setContentView(R.layout.activity_main);  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {  
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);  
            return insets;  
        });  
  
        textView = findViewById(R.id.textview);  
  
        try {  
            JSONObject root = new JSONObject();  
            JSONArray users = new JSONArray();  
  
            root.put("name", "Wow");  
            root.put("age", "It kinda");  
            root.put("email", "Works");  
            // Создаем файл  
            File file = new File(getFilesDir(), "data.json");  
            FileWriter writer = new FileWriter(file);  
            writer.write(root.toString());  
            writer.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
  
        String contents = "";  
        try (FileInputStream fis = this.openFileInput("data.json")) {  
            InputStreamReader inputStreamReader = new  
                InputStreamReader(fis, StandardCharsets.UTF_8);  
            StringReader stringReader = new StringReader("");  
            try (BufferedReader reader = new
```

Рисунок 2.2.2 – Часть 1 описания главной activity



```
        try (BufferedReader reader = new  
            BufferedReader(inputStreamReader)) {  
                String line = reader.readLine();  
                while (line != null) {  
                    stringBuilder.append(line).append('\n');  
                    line = reader.readLine();  
                }  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
        contents = stringBuilder.toString();  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
    }  
  
    Gson gson = new Gson();  
  
    User user = new User();  
  
    try {  
        user = gson.fromJson(contents, User.class);  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
    }  
  
    textView.setText(textView.getText() + "Name: " + user.name +  
        "\nAge: " + user.age + "\nEmail: " + user.email);  
    }  
}
```

Рисунок 2.2.3 - Часть 2 описания главной activity

На рисунках 2.2.2 - 2.2.3 также описан способ отображения сохранённых данных в JSON файле на экран (рисунок 2.2.4).

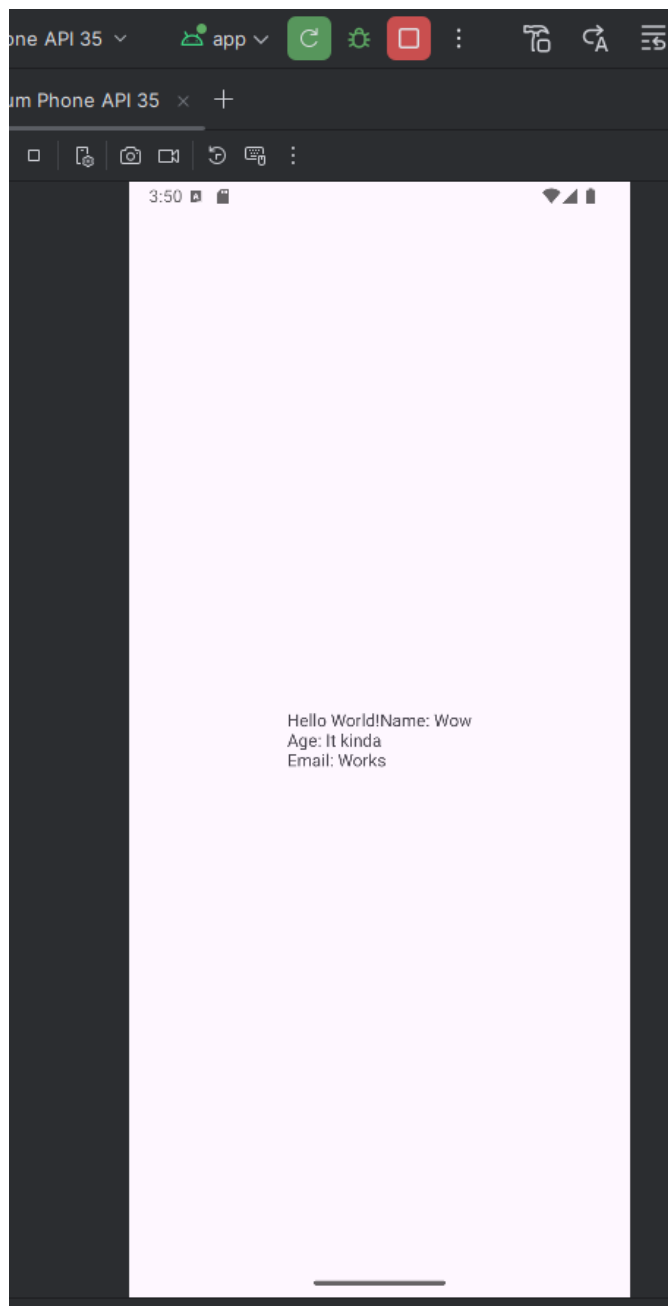


Рисунок 2.2.4 – Отображение итогового вида приложения

Отображение верно, следовательно, приложение работает корректно.

ЗАКЛЮЧЕНИЕ

В ходе данной работы были получены знания по взаимодействию с JSON файлами и провайдером контента. Полученные знания были закреплены путём выполнения практического задания.